FILE HANDLING UTILITY USING JAVA

1. Abstract

File handling is an important feature of Java that enables programs to store and retrieve data from files. This project demonstrates the implementation of a File Handling Utility using Java. The application performs basic file operations such as writing data to a file, reading data from a file, and appending additional data to an existing file. The project uses Java I/O classes such as FileWriter, BufferedWriter, FileReader, and BufferedReader. This utility helps in understanding how data can be permanently stored and managed using files.

2. Objective

The main objectives of this project are:

- To understand file handling concepts in Java.
- To learn how to write data into a file.
- To learn how to read data from a file.
- To learn how to append data to an existing file.
- To understand Java Input/Output (I/O) streams.

3. Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB or above
- Storage: 500 MB free space

Software Requirements

- Operating System: Windows/Linux/MacOS
- Java Development Kit (JDK 8 or above)
- Command Prompt (CMD)
- Notepad or any text editor

4. Introduction

File handling in Java allows programs to create, read, write, and modify files stored on a computer. It is widely used in real-world applications such as banking systems, student management systems, inventory management systems, and log management systems. Java provides various classes in the java.io package to perform these operations efficiently.

In this project, a simple utility is developed to demonstrate the fundamental file operations using Java.

5. Concepts Used

FileWriter

Used to write character data into a file.

BufferedWriter

Provides efficient writing by buffering characters before writing them to the file.

FileReader

Used to read character data from a file.

BufferedReader

Used to read text from a file line by line efficiently.

Exception Handling

Used to handle runtime errors such as file not found or input/output exceptions.

6. Algorithm

Write Operation

1. Create a file.
2. Open FileWriter and BufferedWriter.
3. Write data into the file.
4. Close the writer.

Read Operation

1. Open FileReader and BufferedReader.
2. Read file content line by line.
3. Display the content.

4. Close the reader.

Append Operation

1. Open FileWriter in append mode.
2. Add new content to the existing file.
3. Close the writer.

7. Source Code

```
import java.io.*;


public class FileHandlingUtility {


   public static void writeFile(String fileName) {
      try {
         BufferedWriter bw = new BufferedWriter(new FileWriter(fileName));


         bw.write("Welcome to Java File Handling Project");
         bw.newLine();
         bw.write("This file demonstrates write operation.");


         bw.close();


         System.out.println("Data written successfully.");


      } catch (IOException e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
```

```
public static void readFile(String fileName) {
    try {
        BufferedReader br = new BufferedReader(new FileReader(fileName));

        String line;

        System.out.println("\nFile Contents:");

        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }

        br.close();

    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
}

public static void appendFile(String fileName) {
    try {
        BufferedWriter bw =
                new BufferedWriter(new FileWriter(fileName, true));

        bw.newLine();
        bw.write("This line is appended later.");

        bw.close();
```

```
            System.out.println("\nData appended successfully.");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {

        String fileName = "sample.txt";

        writeFile(fileName);

        readFile(fileName);

        appendFile(fileName);

        System.out.println("\nAfter Appending:");

        readFile(fileName);
    }
}
```

8. Output

Data written successfully.

File Contents:
Welcome to Java File Handling Project
This file demonstrates write operation.

Data appended successfully.

After Appending:

File Contents:
Welcome to Java File Handling Project
This file demonstrates write operation.
This line is appended later.

(Attach the screenshot of the program execution here.)

9. Applications

- Student Record Management Systems
- Banking Applications
- Employee Management Systems
- Log File Maintenance
- Inventory Management Systems
- Data Storage Applications

10. Advantages

- Easy to store and retrieve data.
- Supports permanent data storage.
- Efficient reading and writing operations.
- Useful in real-world applications.
- Provides structured data management.

11. Limitations

- Suitable only for text-based file operations.
- Performance may decrease for very large files.
- Requires proper exception handling.

12. Conclusion

The File Handling Utility project successfully demonstrates the implementation of basic file operations in Java. The program performs writing, reading, and appending operations using Java I/O classes. This project provides a strong foundation for understanding file management techniques that are widely used in software development and enterprise applications.

13. Viva Questions and Answers

Q1. What is file handling?

File handling is the process of creating, reading, writing, and modifying files using programming languages.

Q2. Which package is used for file handling in Java?

The java.io package.

Q3. What is FileWriter?

FileWriter is used to write character data into a file.

Q4. What is FileReader?

FileReader is used to read character data from a file.

Q5. What is BufferedReader?

BufferedReader is used to read text efficiently line by line.

Q6. What is BufferedWriter?

BufferedWriter is used to write text efficiently by buffering output.

Q7. What is append mode?

Append mode adds new data to the end of an existing file without deleting previous data.

Q8. What is IOException?

IOException is an exception that occurs during input/output operations.

Q9. Why is exception handling important?

It prevents program crashes and helps manage errors gracefully.

Q10. What is the difference between FileReader and BufferedReader?

FileReader reads characters directly from a file, while BufferedReader reads data more efficiently using a buffer and supports line-by-line reading.